

Bruise Segmentation Pipeline Knowledge and Defense Document

Architectures, Internal Model Mechanics, Training, Distillation, Thresholding,
Evaluation, Benchmarking, Fairness, and Audit Notes

Generated from the uploaded scripts, logs, benchmark CSVs, fairness outputs, and fixed-test
evaluation outputs.

Purpose. This document is written as a teaching and defense guide. It explains what was done, why each step exists, how to answer questions, and which conclusions are safe to make from the current results.

July 9, 2026

Contents

1	Executive summary	5
1.1	The safest one-paragraph story	5
1.2	Current model ranking from the uploaded results	5
2	Problem setup: what the pipeline is solving	7
2.1	Task definition	7
2.2	Input and output	7
2.3	Why this is a medical/clinical-style segmentation problem	8
3	Dataset design and leakage protection	8
3.1	Fixed test set	8
3.2	Subject-level leakage	8
3.3	Skin-tone coverage	8
4	End-to-end pipeline overview	9
5	Architectures: every model family in the uploaded pipeline	10
5.1	SegFormer architecture	10
5.1.1	SegFormer-B2 teacher	10
5.1.2	SegFormer-B0 direct	11
5.1.3	SegFormer-B0 distilled	11
5.1.4	Fair KD Approach A	11
5.1.5	Fair KD Approach B	11
5.1.6	ALS-to-WL distillation	11
5.2	YOLO semantic segmentation models	11
5.2.1	YOLO direct	11
5.2.2	YOLO distilled	12
5.3	Models not present in this uploaded result set	12
6	Model internals in detail: what happens inside each architecture	12
6.1	SegFormer internals	13
6.1.1	Step 1: input tensor and normalization	13
6.1.2	Step 2: overlapping patch embedding	13
6.1.3	Step 3: Mix Transformer encoder	14
6.1.4	Step 4: efficient self-attention	14
6.1.5	Step 5: Mix-FFN and why convolution still appears in a transformer	14

6.1.6	Step 6: MLP decoder head	14
6.1.7	B0 vs B2 internally	15
6.2	SegFormer-B2 teacher internals	15
6.3	SegFormer-B0 direct internals	15
6.4	SegFormer-B0 distilled internals	16
6.5	Fairness-aware SegFormer internals	16
6.5.1	Fair KD Approach A: ITA-reweighted distillation	17
6.5.2	Fair KD Approach B: feature alignment using MMD	17
6.6	ALS-to-WL distillation internals	17
6.7	YOLO semantic segmentation internals	18
6.7.1	YOLO forward pass	18
6.7.2	Native YOLO path	18
6.7.3	Custom raw-logit path	19
6.8	YOLO direct vs YOLO distilled internals	19
6.9	DeepLabV3+ MobileNetV3 internals	20
6.9.1	MobileNetV3 backbone	20
6.9.2	ASPP context module	20
6.9.3	DeepLabV3+ decoder	20
6.10	How logits, probabilities, and masks differ internally	21
6.11	One-image forward pass: what happens from camera frame to overlay	21
7	Training losses	22
7.1	Supervised segmentation loss: Dice + BCE	22
7.2	Knowledge distillation loss	22
7.3	Why soft labels help	23
7.4	Fairness-aware loss	23
8	Temperature scaling	23
8.1	What temperature scaling does	23
8.2	Why teacher temperature is useful	24
8.3	Why YOLO temperature results are currently under audit	24
9	Thresholding	25
9.1	Why thresholding is necessary	25
9.2	How threshold affects precision and recall	25
9.3	Validation threshold search protocol	26
10	Evaluation metrics	26

10.1 Pixel counts	26
10.2 Dice	27
10.3 IoU	27
10.4 Precision and recall	27
10.5 Zero Dice and complete miss	27
10.6 Surface Dice, ASD, and HD95	28
11 Evaluation resolution and resizing	28
11.1 Why images and masks are resized	28
11.2 Why nearest-neighbor is used for masks	28
11.3 Evaluation vs deployment resizing	29
12 Benchmarking	29
12.1 Raw forward vs full pipeline	29
12.2 Why disk reading is excluded	30
12.3 Why warmup and CUDA synchronization are needed	30
12.4 fp32 vs fp16	31
13 Result interpretation	33
13.1 Main accuracy result	33
13.2 Why B0 distilled is the best deployment answer	33
13.3 Why B2 teacher is still important	33
13.4 Why YOLO should be discussed carefully	34
14 Fairness and ITA evaluation	34
14.1 What fairness means in this project	34
14.2 Fairness gap	34
14.3 How to answer fairness questions	35
15 Audit findings and what should be fixed before final publication	35
15.1 What looks solid	35
15.2 What needs caution	36
15.3 Exact YOLO fix	36
16 How to present the results	36
16.1 Best 60-second explanation	36
16.2 Best 15-second conclusion	36
17 Script map	37

18 Q&A defense guide

18.1 Architecture and model-internal Q&A

19 Final recommendations

19.1 Use this as the final model conclusion for now

19.2 Do this before calling the whole project final

19.3 Canonical final table to show

20 Appendix: formulas in one place

38

43

45

45

45

46

1 Executive summary

This project is a binary semantic segmentation pipeline for bruises in white-light (WL) skin images. The input is an RGB image. The output is a binary mask that marks bruise pixels. The final deployment goal is not just a score; it is a mask that can be resized back to the original camera image and overlaid on the image.

Main result. The strongest current deployment choice is **SegFormer-B0 distilled**. It has the highest fixed-test median Dice in the uploaded 640 evaluation and runs much faster than the SegFormer-B2 teacher in the full deployment pipeline.

Important audit finding. The YOLO raw-logit temperature/threshold evaluation path is not yet fully trustworthy. The native Ultralytics YOLO path and the custom raw-logit path give very different Dice values. The likely reason is a preprocessing mismatch: the raw-logit scripts appear to use the generic ImageNet-normalized `BruiseDataset`, while YOLO expects images scaled to 0–1. YOLO should be rerun with a YOLO-specific preprocessing path before making final accuracy or fairness claims about YOLO.

1.1 The safest one-paragraph story

We trained a high-capacity SegFormer-B2 teacher and smaller deployment-oriented students. The B0 distilled student learned from both ground-truth masks and the B2 teacher’s soft probability maps. On the fixed 185-image test set, B0 distilled reached the best median Dice while running more than twice as fast as B2 in the full camera-to-mask benchmark. Fairness-aware distillation reduced the skin-tone performance gap in some variants, especially Fair KD B, but B0 distilled remains the best overall accuracy-speed point. YOLO remains under audit because the raw-logit evaluation path likely used incorrect preprocessing.

1.2 Current model ranking from the uploaded results

Table 1: Main fixed-test and full-pipeline benchmark summary. All test metrics are from the 185-image fixed test set at 640x640 using validation-selected thresholds. FPS is fp32 full-pipeline FPS.

Model	Thr.	Med. Dice	Mean Dice	Med. IoU	Med. Prec.	Med. Rec.	Miss %	FPS
SegFormer-B0 distilled	0.55	0.828	0.787	0.707	0.917	0.802	0.54	86.2
SegFormer-B2 teacher	0.75	0.827	0.767	0.705	0.961	0.760	0.00	33.6
Fair KD A	0.30	0.825	0.768	0.701	0.922	0.783	1.08	86.2
Fair KD B	0.35	0.824	0.779	0.700	0.930	0.753	0.00	85.5
ALS->WL B0 distill	0.35	0.811	0.754	0.682	0.925	0.771	0.00	85.6
SegFormer-B0 direct	0.60	0.811	0.775	0.681	0.914	0.766	0.54	86.5
YOLO distilled	0.40	0.726	0.608	0.570	0.786	0.800	9.73	50.4
YOLO direct	0.10	0.647	0.506	0.478	0.799	0.668	19.46	51.8

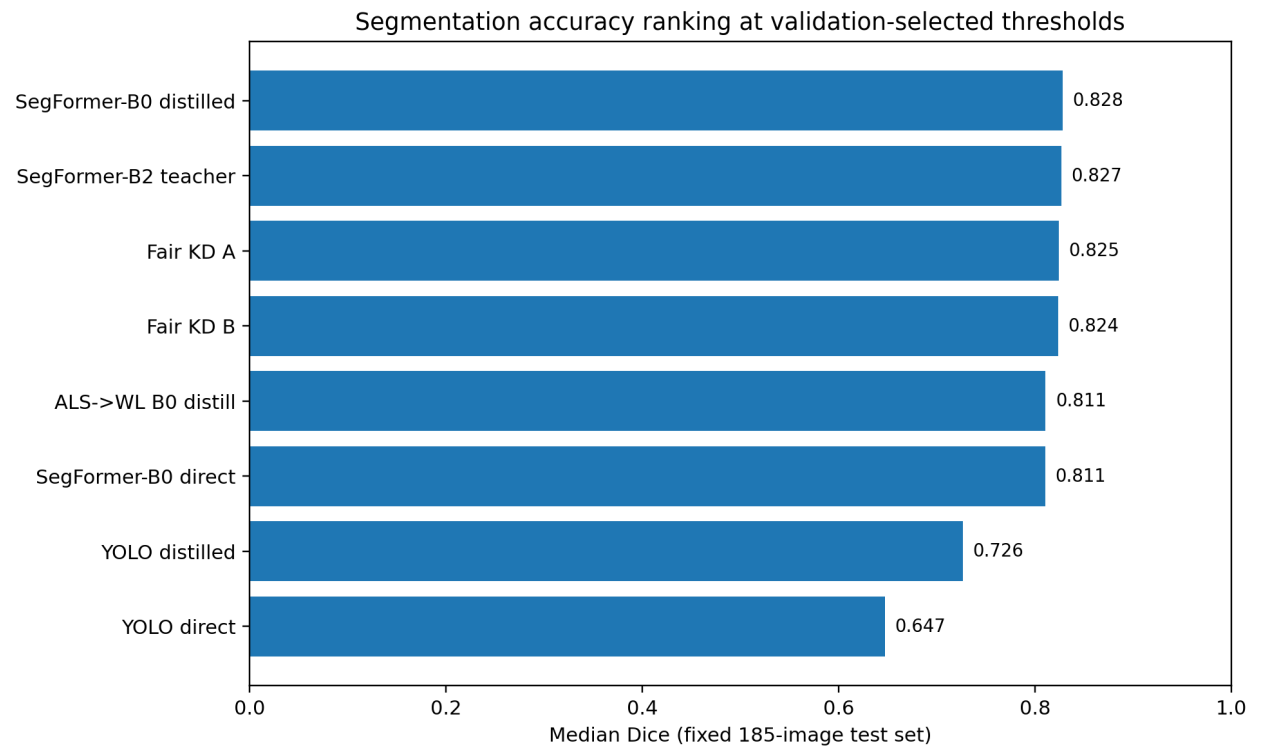


Figure 1: Accuracy ranking by median Dice. Median is emphasized because segmentation results are often skewed by hard images and complete misses.

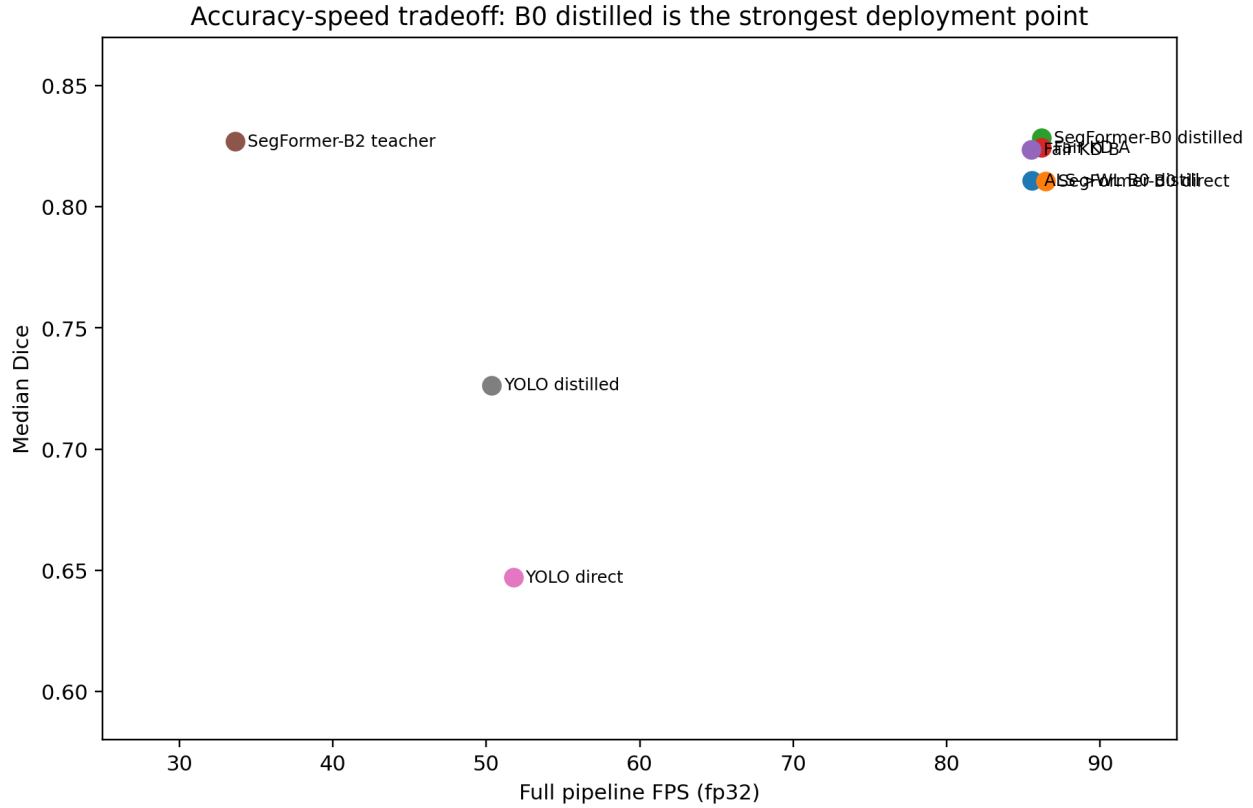


Figure 2: Accuracy-speed tradeoff. The best deployment model should be high on Dice and far to the right on FPS. SegFormer-B0 distilled is the cleanest point from the current uploaded outputs.

2 Problem setup: what the pipeline is solving

2.1 Task definition

The pipeline solves **binary semantic segmentation**. For each image, the model predicts a binary mask:

$$\hat{Y}_{ij} = \begin{cases} 1, & \text{pixel } (i, j) \text{ predicted as bruise,} \\ 0, & \text{pixel } (i, j) \text{ predicted as background.} \end{cases}$$

The model does not merely classify the whole image as bruise/no bruise. It predicts a pixel-level map. This is important because bruise shape, area, and boundary quality matter.

2.2 Input and output

- **Input:** a WL RGB image.
- **Model tensor size:** the current evaluation uses 640x640 tensors.
- **Raw model output:** a logit map, usually one channel for binary segmentation.
- **Probability map:** obtained by sigmoid for SegFormer, and by the YOLO logit-to-probability helper for YOLO.
- **Binary mask:** obtained by applying the validation-selected threshold.

- **Deployment output:** the binary mask is resized back to the original image resolution for overlay.

2.3 Why this is a medical/clinical-style segmentation problem

Bruises can vary in size, color, shape, skin tone visibility, lighting, and boundary clarity. This makes simple classification insufficient. The model needs to identify *where* the bruise is, not only whether one exists. That is why overlap metrics such as Dice and IoU, plus miss rate and boundary metrics, are needed.

3 Dataset design and leakage protection

3.1 Fixed test set

The uploaded evaluation outputs use a **fixed 185-image test set**. This set is treated as locked. It should not be used to select hyperparameters, thresholds, alpha, or model choices during training. It is only for final evaluation.

A strong answer if asked: **The test set is not used to tune threshold.** We tune thresholds on validation and then apply the fixed threshold once on the test set.

3.2 Subject-level leakage

For medical imaging or subject-specific skin images, leakage can occur when images from the same subject appear in both training and testing. The ITA verification log reports a leakage check pass: zero subject overlap between the train/validation split and the fixed test manifest. That is important because subject leakage can artificially inflate results.

3.3 Skin-tone coverage

The ITA pipeline computes skin-tone group labels. The available coverage report indicates the WL fixed test set has 185 valid ITA labels and 0 unclassified images. The train/validation split had 829 valid ITA rows out of 831, with 2 unknown/unclassified.

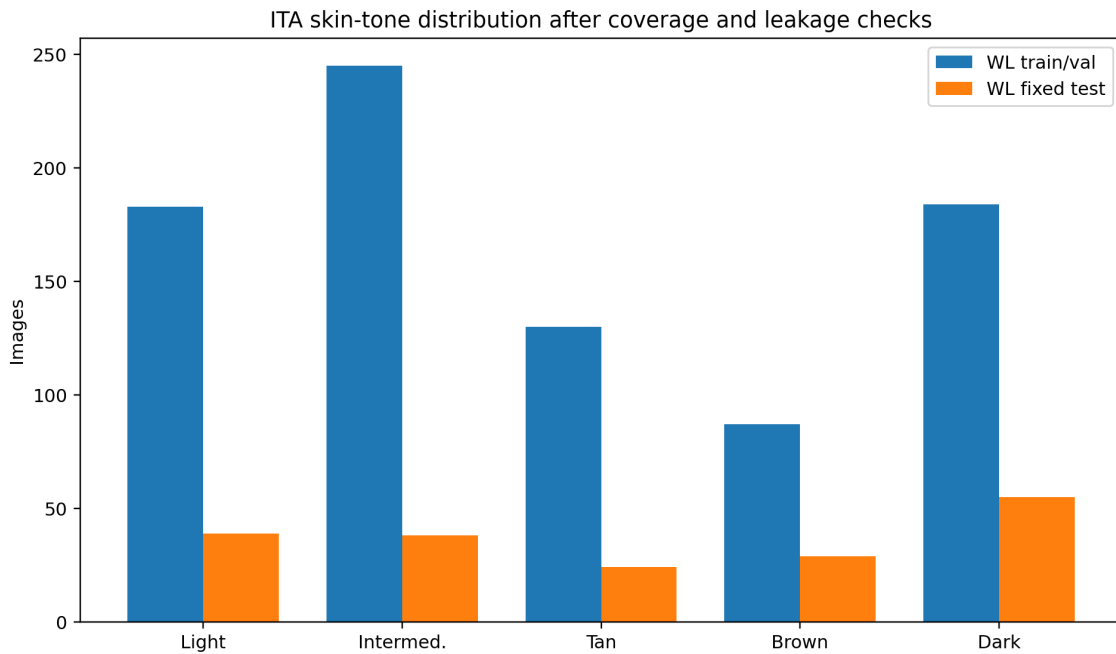


Figure 3: ITA distribution for WL train/validation and fixed test images. The test set is not perfectly balanced but includes all major ITA groups except Very Light as a separate category.

4 End-to-end pipeline overview

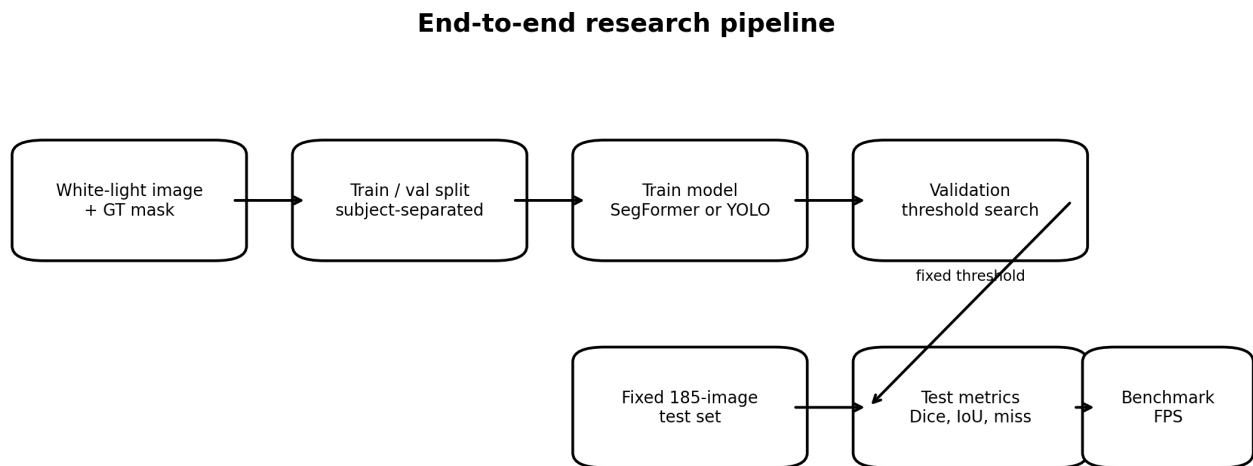


Figure 4: High-level research pipeline: split data, train model, select threshold on validation, evaluate once on fixed test, and benchmark deployment speed.

The pipeline has five conceptual stages:

1. **Data preparation:** read image/mask paths, normalize manifests, create train/validation/test splits,

and verify subject leakage.

2. **Model training:** train direct, distilled, fairness-aware, or ALS-to-WL models.
3. **Validation threshold search:** sweep candidate thresholds on validation data only.
4. **Fixed-test evaluation:** evaluate Dice, IoU, recall, precision, surface metrics, miss rate, and zero-Dice rate.
5. **Deployment benchmark:** time both raw forward pass and full camera-to-overlay pipeline.

5 Architectures: every model family in the uploaded pipeline

5.1 SegFormer architecture

SegFormer is a transformer-based semantic segmentation model. It has two major parts:

- **Mix Transformer encoder:** extracts multi-scale visual features from the input image.
- **MLP decoder head:** fuses the multi-scale features and produces dense segmentation logits.

For this project, the standard multi-class segmentation head is replaced/adapted into a **one-channel binary segmentation head**. One logit per pixel is produced. Applying sigmoid converts each logit into a bruise probability.

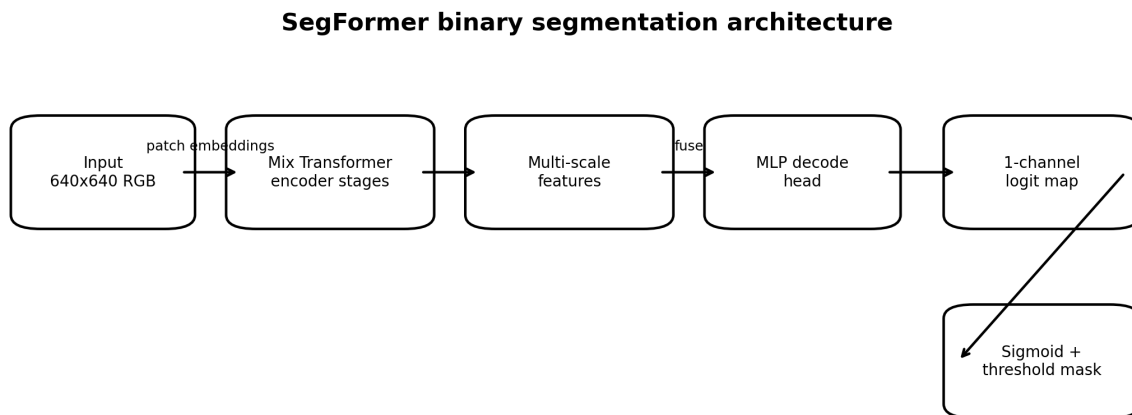


Figure 5: SegFormer binary segmentation flow. The wrapper upsamples logits to match the input tensor size before evaluation.

5.1.1 SegFormer-B2 teacher

The B2 teacher is the larger SegFormer model in this experiment. It has about 27.35M parameters in the uploaded consolidated output. It is used as the high-capacity reference model. Its job is to learn strong segmentation behavior and provide soft labels for smaller students.

Why B2 teacher matters:

- larger model capacity;
- better contextual representation;
- useful soft probability maps for distillation;
- high accuracy but slower deployment speed.

5.1.2 SegFormer-B0 direct

B0 direct is the small SegFormer student trained directly against ground-truth masks. It does not use teacher probabilities. It provides the clean baseline for asking: *does distillation improve the small model?*

5.1.3 SegFormer-B0 distilled

B0 distilled uses both the ground-truth mask and the B2 teacher soft probability map. This is the most important deployment model in the current results. It keeps the B0 speed advantage while improving typical Dice over B0 direct.

5.1.4 Fair KD Approach A

Fair KD A adds ITA-aware reweighting to distillation. The idea is that if a skin-tone group has weaker validation performance, its distillation loss gets a larger weight. This attempts to force the student to pay more attention to underperforming groups.

5.1.5 Fair KD Approach B

Fair KD B adds feature alignment using Maximum Mean Discrepancy (MMD). Instead of matching only final output probability maps, it also encourages the student's internal feature distribution to match the teacher's feature distribution within ITA groups.

5.1.6 ALS-to-WL distillation

The ALS-to-WL model transfers information from an ALS-input teacher into a WL student. The motivation is cross-modality knowledge transfer: ALS may reveal bruise information that is harder to see in WL, and the student tries to inherit some of that knowledge while still accepting WL images at deployment.

5.2 YOLO semantic segmentation models

The uploaded pipeline includes YOLO semantic direct and YOLO semantic distilled models. YOLO is normally known for fast detection/segmentation style pipelines. In this project, the YOLO model is used to produce segmentation outputs that can be converted to a bruise probability mask.

5.2.1 YOLO direct

YOLO direct is trained directly against bruise masks. In the native Ultralytics path, the YOLO direct model performs much better than the raw-logit temperature-scaled path. This discrepancy is the reason for the audit warning.

5.2.2 YOLO distilled

YOLO distilled is a student model trained with teacher signal. In the uploaded 640 raw-logit evaluation, it improves over YOLO direct, but these YOLO numbers should not be treated as final until the preprocessing path is corrected.

YOLO caveat. The benchmark script uses YOLO-style preprocessing: resize, tensor conversion, divide by 255. That is correct for YOLO timing. However, the raw-logit threshold/evaluation scripts use the generic **BruiseDataset**, which applies ImageNet normalization. This mismatch likely explains why raw-logit YOLO accuracy is much lower than native Ultralytics accuracy.

5.3 Models not present in this uploaded result set

Earlier project discussions mentioned DeepLabV3+ MobileNetV3. The current uploaded scripts/results used for this document do not include final DeepLabV3+ evaluation CSVs. Therefore this document does not make numerical claims about DeepLab. If DeepLab is added back, it should be evaluated using the same fixed test set, validation threshold selection, full-pipeline benchmark definition, and fairness protocol.

6 Model internals in detail: what happens inside each architecture

This section is the missing “inside the model” explanation. It explains the internal computation path, not just the model names. A good way to understand every model is to separate it into five parts:

1. **Preprocessing:** convert an image into the tensor distribution the model expects.
2. **Encoder/backbone:** convert pixels into feature maps.
3. **Context module / neck:** mix local and global information.
4. **Decoder/head:** convert features back into pixel-level logits.
5. **Postprocessing:** convert logits into probability maps and binary masks.

For all segmentation models, the key idea is the same: the model starts with image pixels and ends with a spatial map. Unlike classification, the spatial grid is preserved or reconstructed so each output location corresponds to a region of the input image.

6.1 SegFormer internals

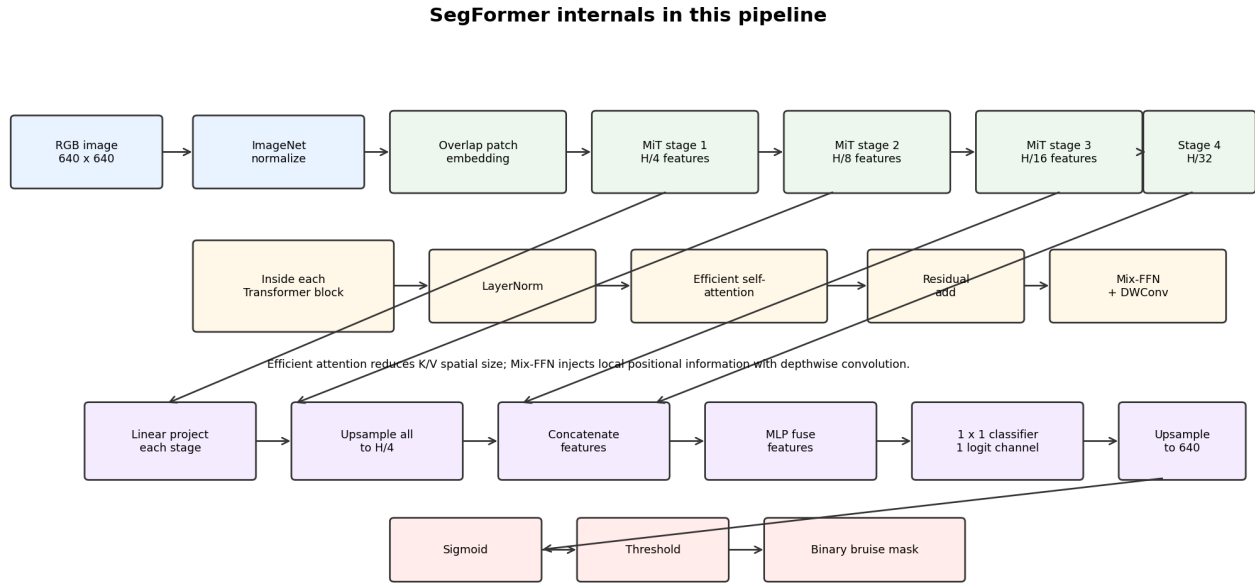


Figure 6: Detailed SegFormer internals. The encoder creates multi-scale features. The decoder projects, upsamples, concatenates, fuses, and classifies them into a one-channel bruise logit map.

6.1.1 Step 1: input tensor and normalization

In your SegFormer path, each image is read as RGB, resized to the training/evaluation resolution, ImageNet-normalized, and converted to a PyTorch tensor:

$$x \in \mathbb{R}^{B \times 3 \times 640 \times 640}.$$

The three channels are red, green, and blue. The batch dimension is B . The SegFormer backbone was pretrained on natural images, so ImageNet-style normalization is appropriate for this path. This is why the generic `BruiseDataset` is correct for SegFormer but suspicious for YOLO raw-logit evaluation.

6.1.2 Step 2: overlapping patch embedding

A normal Vision Transformer splits an image into non-overlapping square patches. SegFormer instead uses **overlapping patch embeddings**. In practice, this behaves like a convolutional projection: neighboring patches share some pixels.

Why this matters for bruises:

- bruise boundaries are soft and irregular;
- non-overlapping patches can create blocky boundary artifacts;
- overlapping patches preserve local continuity better.

After patch embedding, the image becomes a sequence or feature grid that can be processed by transformer blocks.

6.1.3 Step 3: Mix Transformer encoder

SegFormer uses a Mix Transformer encoder, usually called MiT. It produces feature maps at several spatial scales. Conceptually, the stages are:

Table 2: Conceptual SegFormer encoder scales. Exact channel counts depend on B0/B2, but the multi-scale structure is the key.

Stage	Spatial scale	Captures	Why useful for bruises
Stage 1	about $H/4 \times W/4$	fine texture and edges	helps boundaries and small bruises
Stage 2	about $H/8 \times W/8$	local shapes	helps medium bruise regions
Stage 3	about $H/16 \times W/16$	larger context	helps separate bruise from lighting/color variation
Stage 4	about $H/32 \times W/32$	global context	helps understand full lesion/background context

Each transformer block roughly follows this pattern:

$$X' = X + \text{Attention}(\text{LN}(X)),$$

$$X'' = X' + \text{MixFFN}(\text{LN}(X')).$$

That means the block normalizes features, applies attention, adds the result back with a residual connection, then applies a feed-forward network and another residual connection.

6.1.4 Step 4: efficient self-attention

Standard self-attention is expensive because every image token attends to every other token. If there are N tokens, full attention costs roughly $O(N^2)$. SegFormer reduces this cost using **spatial reduction attention**. In simple words:

- queries still represent the current feature positions;
- keys and values are spatially reduced before attention;
- the model gets long-range context without the full cost of global attention at high resolution.

A useful answer if asked: **attention lets each region use information from other image regions**. For bruises, this can help distinguish true bruise color from shadows, skin folds, illumination changes, or artifacts.

6.1.5 Step 5: Mix-FFN and why convolution still appears in a transformer

The feed-forward part of a transformer usually operates independently at each token. SegFormer’s Mix-FFN includes a depthwise convolution inside the feed-forward network. This gives the model local spatial awareness without needing explicit positional embeddings.

This matters because segmentation needs location-sensitive decisions. A pixel is not classified in isolation; it should agree with its neighboring pixels.

6.1.6 Step 6: MLP decoder head

The decoder receives feature maps from all four encoder stages. The decoder does the following:

1. linearly projects each stage’s feature channels to a common decoder dimension;
2. upsamples all features to the same spatial scale;
3. concatenates them;
4. fuses them with an MLP/convolutional fusion layer;
5. applies a classifier layer to produce one bruise logit channel.

Your wrapper then upsamples the output logits to the input tensor resolution if needed:

$$z_S \in \mathbb{R}^{B \times 1 \times 640 \times 640}.$$

Finally:

$$p = \sigma(z_S), \quad \hat{Y}_{ij} = \mathbb{I}[p_{ij} \geq \tau].$$

6.1.7 B0 vs B2 internally

Both B0 and B2 use the same overall SegFormer design, but B2 is deeper/wider. B2 has more channels and more transformer blocks, so it can represent more complex patterns but requires more compute.

Table 3: High-level B0 vs B2 interpretation. Parameter counts are from the uploaded consolidated outputs.

Variant	Internal meaning	Practical effect	Your pipeline role
B0	small MiT encoder + small decoder	fast, fewer parameters	deployment student / direct baseline
B2	larger MiT encoder + larger decoder	stronger but slower	teacher model

A strong defense answer: **B2 is not better because it is magically different; it is the same family with more capacity. B0 distilled is valuable because it keeps the small B0 runtime while learning some of the teacher’s behavior.**

6.2 SegFormer-B2 teacher internals

The teacher is a normal SegFormer-B2 binary segmentation model during supervised training. Internally:

1. input image goes through the B2 MiT encoder;
2. decoder produces a bruise logit map;
3. Dice-BCE loss compares that logit map to ground truth;
4. after training, the best checkpoint is frozen;
5. during distillation, the teacher generates probability maps, not gradients.

The teacher function in the pipeline runs under `torch.no_grad()` so the teacher does not update. This saves GPU memory and prevents the student training from changing the teacher.

6.3 SegFormer-B0 direct internals

B0 direct has the same forward computation type as B2 but with smaller internal capacity. It is trained only with ground-truth masks:

$$L_{direct} = L_{sup} = BCE(z, Y) + (1 - Dice(\sigma(z), Y)).$$

This model answers the question: *How good is the small model without teacher help?*

6.4 SegFormer-B0 distilled internals

B0 distilled has the same architecture as B0 direct. The difference is the training signal. During each batch:

1. the frozen B2 teacher predicts a softened probability map P_T ;
2. the B0 student predicts logits z_S ;
3. supervised loss compares z_S to the ground-truth mask;
4. KD loss compares z_S to the teacher probability map;
5. both losses are blended using alpha.

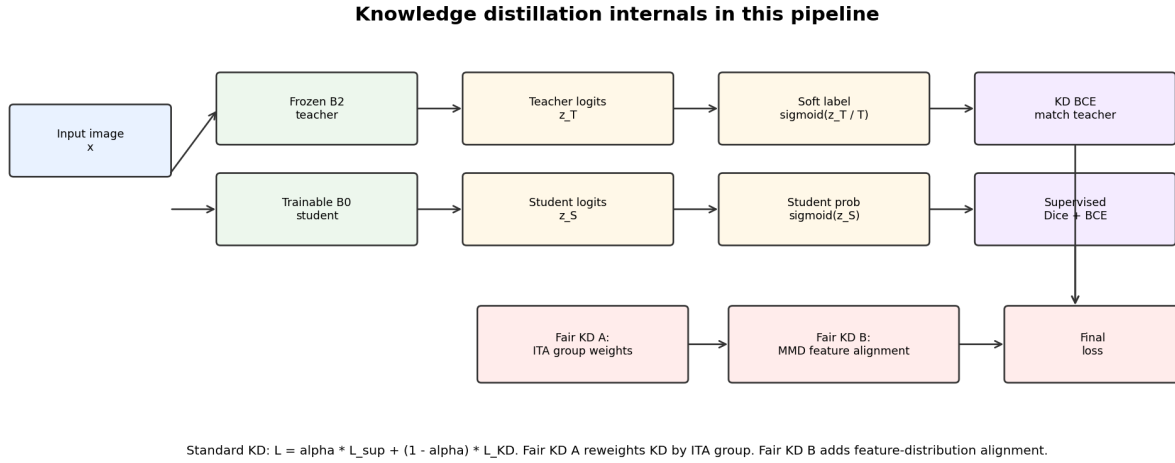


Figure 7: Internal flow of standard and fairness-aware knowledge distillation in the uploaded pipeline.

The core loss is:

$$L = \alpha L_{sup}(z_S, Y) + (1 - \alpha) BCE(z_S, P_T).$$

This is important: the student is not simply trained on the teacher. It still sees the actual ground-truth mask. The teacher provides extra soft guidance.

6.5 Fairness-aware SegFormer internals

The fairness models are still SegFormer-B0 students. They do not use a different architecture at inference time. The difference is in the training loss.

6.5.1 Fair KD Approach A: ITA-reweighted distillation

Approach A changes the KD term by assigning a weight to each skin-tone group. Groups with lower validation Dice receive a higher KD weight:

$$w_g = \frac{1}{Dice_g + \epsilon},$$

then weights are normalized so the average weight is around 1. The loss becomes:

$$L = \alpha L_{sup} + (1 - \alpha) \cdot \text{mean}_i[w_{g_i} \cdot BCE(z_{S,i}, P_{T,i})].$$

The supervised ground-truth term is not reweighted in the uploaded implementation; the fairness emphasis is applied to the teacher-imitation term. The goal is to make the student copy the teacher more carefully on groups where the student is currently weak.

6.5.2 Fair KD Approach B: feature alignment using MMD

Approach B adds another internal constraint. Instead of matching only output probabilities, it also tries to match teacher and student **feature distributions** within ITA groups.

The implementation captures decoder features using forward hooks. Then it:

1. global-average-pools decoder features from $[B, C, h, w]$ to $[B, C]$;
2. L2-normalizes feature vectors;
3. groups them by ITA group;
4. computes RBF-kernel Maximum Mean Discrepancy (MMD) between teacher and student features;
5. adds $\lambda_{mmd}MMD$ to the loss.

Conceptually:

$$L_B = L_A + \lambda_{mmd} \frac{1}{G} \sum_g MMD^2(T_g, S_g).$$

A strong answer if asked: **Approach B is more internal than Approach A. Approach A changes how strongly the student copies teacher outputs per group. Approach B tries to make the student’s hidden representation distribution look like the teacher’s distribution per group.**

6.6 ALS-to-WL distillation internals

The ALS-to-WL model is also a SegFormer-B0 student at deployment. It accepts WL images. The difference is that the teacher comes from an ALS-input model. The idea is cross-modality transfer:

- ALS may contain bruise cues that are stronger or different from WL;
- the ALS teacher learns from that modality;
- the WL student is trained to inherit some of the ALS teacher’s decision structure;
- deployment remains WL-only, so the final model can run on ordinary WL inputs.

The important defense point is that ALS-to-WL distillation is not changing the inference input. The deployed student still uses WL. ALS is used as a source of teacher knowledge during training.

6.7 YOLO semantic segmentation internals

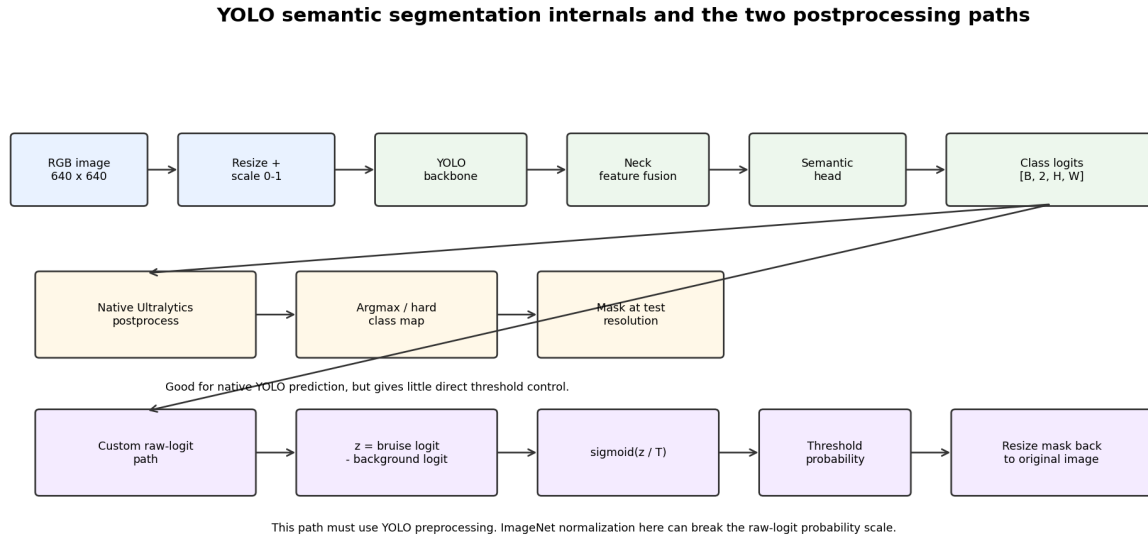


Figure 8: YOLO semantic internals and the two different postprocessing routes. The audit issue is specifically about the custom raw-logit route needing YOLO-style preprocessing.

YOLO in this project is trained through the Ultralytics semantic segmentation training path. It is different from SegFormer in several ways:

- it uses Ultralytics’ own training loop;
- masks are stored as class-index PNGs, where 0 is background and 1 is bruise;
- training uses YOLO’s optimizer/scheduler/augmentation setup, not the custom SegFormer trainer;
- the semantic head outputs class logits over spatial positions.

6.7.1 YOLO forward pass

Conceptually, a YOLO semantic model has:

1. **Backbone:** extracts increasingly abstract visual features.
2. **Neck:** fuses features across scales so small and large objects can both be represented.
3. **Semantic head:** emits dense class logits for every output location.

In a two-class semantic setup, the raw output can be interpreted as:

$$Z \in \mathbb{R}^{B \times 2 \times H \times W},$$

where channel 0 is background logit and channel 1 is bruise logit.

6.7.2 Native YOLO path

The native Ultralytics path performs its own postprocessing and returns a hard predicted semantic mask. This path gave much stronger YOLO direct results than the custom raw-logit path in the audit. That

means YOLO itself may not be weak; the custom evaluation route may not be equivalent to native inference.

6.7.3 Custom raw-logit path

The custom YOLO raw-logit path bypasses native postprocessing. It extracts raw class logits and computes bruise probability as:

$$p_{bruise} = \sigma \left(\frac{z_{bruise} - z_{background}}{T} \right).$$

This is mathematically equivalent to two-class softmax when $T = 1$:

$$\frac{e^{z_1}}{e^{z_0} + e^{z_1}} = \sigma(z_1 - z_0).$$

Then the pipeline thresholds the probability map:

$$\hat{Y}_{ij} = \mathbb{K}[p_{bruise,ij} \geq \tau].$$

This path is attractive because it gives threshold and temperature control. However, it is only valid if the input preprocessing matches how YOLO was trained. YOLO expects resized images scaled to 0–1. If ImageNet normalization is accidentally applied, the model receives a different input distribution and raw logits are not comparable.

Important defense wording: Do not say “YOLO failed.” Say: “The current custom raw-logit YOLO evaluation path is under audit because it likely used the wrong preprocessing. Native YOLO was stronger, so YOLO needs a corrected rerun before final comparison.”

6.8 YOLO direct vs YOLO distilled internals

YOLO direct is trained on class-index ground-truth masks. YOLO distilled is different because it uses offline pseudo-mask distillation.

For YOLO distilled, the teacher probability map is fused with the ground truth before Ultralytics training:

$$fused = \alpha GT + (1 - \alpha) P_T.$$

Then the fused soft map is converted to a hard class-index pseudo-mask:

$$pseudo = \mathbb{K}[fused \geq pseudo_threshold].$$

Why offline? Because the Ultralytics trainer does not expose the same simple per-batch teacher-loss hooks as the custom SegFormer trainer. So instead of computing teacher loss online during every YOLO batch, the pipeline builds a pseudo-mask dataset before training.

A useful comparison:

Table 4: Online vs offline distillation in this project.

Aspect	SegFormer distillation	YOLO distillation
Teacher used during batch training?	Yes, online teacher probability	No, pseudo-masks built before training
Student loss sees soft probabilities?	Yes, BCE against teacher probability	Indirectly, through fused hard pseudo-mask
Trainer	Custom PyTorch trainer	Ultralytics trainer
Reason	Full control over loss	Ultralytics loop is less direct to modify

6.9 DeepLabV3+ MobileNetV3 internals

DeepLabV3+ MobileNetV3 internals (architecture explanation, no final uploaded metrics)

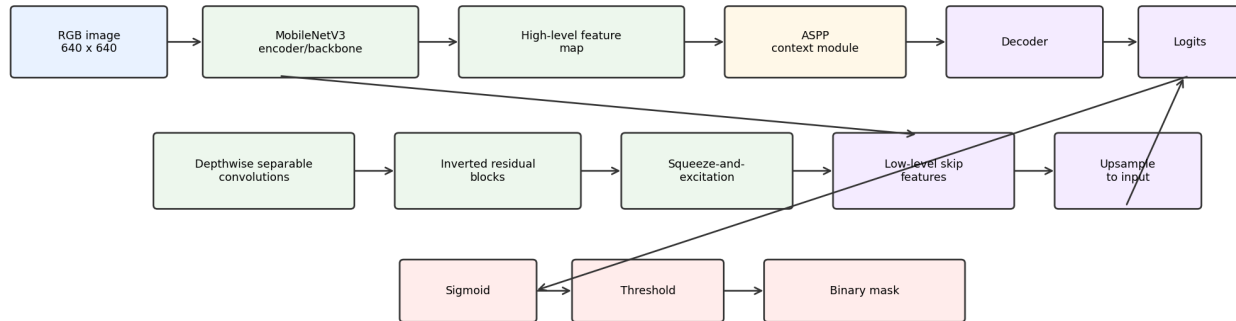


Figure 9: DeepLabV3+ MobileNetV3 architecture internals. This architecture is explained for defense knowledge, but the uploaded final result set did not include final DeepLab metrics.

DeepLabV3+ MobileNetV3 was discussed as part of the broader model family. The final uploaded CSVs used for this PDF do not include final DeepLab metrics, so this document does not rank it numerically. Still, you should understand it in case someone asks.

6.9.1 MobileNetV3 backbone

MobileNetV3 is a lightweight convolutional backbone. Its purpose is to extract features with fewer parameters and fewer operations than a large CNN or transformer. It uses ideas such as:

- **Depthwise separable convolution:** split spatial filtering and channel mixing to reduce computation.
- **Inverted residual blocks:** expand channels, process with cheap depthwise convolution, then project back down.
- **Squeeze-and-excitation:** learn which channels are important and reweight them.
- **Efficient activations:** approximate expensive nonlinearities with cheaper versions.

6.9.2 ASPP context module

DeepLabV3+ uses Atrous Spatial Pyramid Pooling (ASPP). Atrous/dilated convolutions look at a wider context without reducing feature-map resolution. ASPP applies several dilation rates in parallel, so the model sees both local and larger context.

For bruise segmentation, ASPP can help because bruise appearance depends on both local color and surrounding skin context.

6.9.3 DeepLabV3+ decoder

The “plus” in DeepLabV3+ refers to the decoder improvement. The decoder combines:

- high-level ASPP features that understand context;
- low-level features that preserve edges and detail;
- upsampling to recover the original segmentation resolution.

If asked why DeepLabV3+ is useful: it is a lightweight convolutional segmentation baseline. If asked why it is not the main conclusion here: the uploaded final results in this audit do not include complete DeepLab metrics, so it should not be ranked against the final SegFormer results unless rerun under the same protocol.

6.10 How logits, probabilities, and masks differ internally

A common confusion is the difference between logit, probability, and mask:

Table 5: Internal output types.

Name	Meaning	Example
Logit	raw model score before sigmoid/softmax	$z = 2.0$
Probability	confidence after sigmoid/softmax	$p = 0.88$
Binary mask	hard decision after threshold	$\hat{y} = 1$ if $p \geq \tau$

A model is trained using differentiable logits/probabilities, but evaluated and deployed as a binary mask. Thresholding is therefore not a cosmetic step; it directly changes precision, recall, Dice, miss rate, and overlay appearance.

6.11 One-image forward pass: what happens from camera frame to overlay

For the selected SegFormer deployment model, one inference pass is:

1. receive RGB image from camera in memory;
2. resize to 640x640;
3. ImageNet-normalize;
4. move tensor to GPU;
5. run SegFormer-B0 distilled forward pass;
6. get one-channel logit map;
7. apply sigmoid to get bruise probability;
8. apply validation-selected threshold, e.g. $\tau = 0.55$;
9. convert to binary mask;
10. resize mask back to original image height and width using nearest-neighbor interpolation;
11. overlay mask on the original camera image.

This is exactly why the full-pipeline benchmark is more meaningful than raw forward FPS for deployment. Raw forward FPS only tests the neural network. Full-pipeline FPS tests the actual camera-to-overlay path.

7 Training losses

7.1 Supervised segmentation loss: Dice + BCE

The supervised SegFormer models use a combined Dice-BCE loss. Binary cross-entropy stabilizes pixel-wise classification, while Dice directly optimizes mask overlap.

Let z_{ij} be the model logit for pixel (i, j) , $p_{ij} = \sigma(z_{ij})$, and $y_{ij} \in \{0, 1\}$ be the ground-truth mask. The supervised loss is:

$$L_{sup} = BCE(z, Y) + \left(1 - \frac{2 \sum_{ij} p_{ij} y_{ij} + s}{\sum_{ij} p_{ij} + \sum_{ij} y_{ij} + s}\right),$$

where s is a smoothing constant.

7.2 Knowledge distillation loss

Knowledge distillation combines ground truth with teacher probabilities:

$$L_{distill} = \alpha L_{sup}(S, Y) + (1 - \alpha) BCE(z_S, P_T),$$

where:

- S is the student model;
- Y is the ground-truth binary mask;
- z_S is the student logit map;
- P_T is the teacher probability map;
- α controls the balance between ground truth and teacher imitation.

Knowledge distillation flow

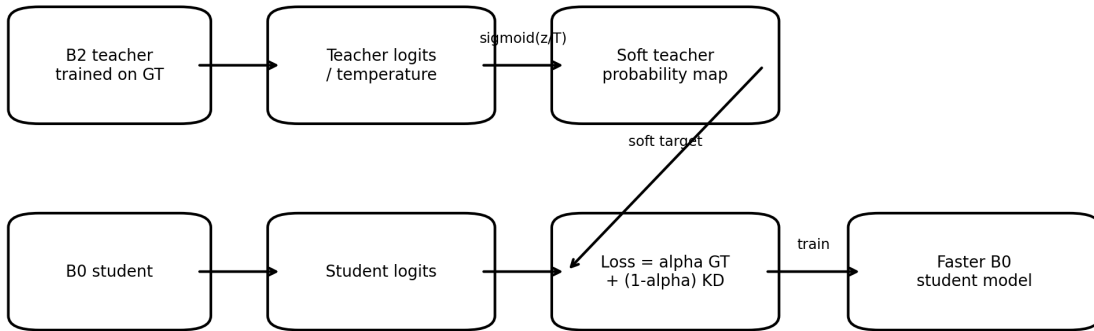


Figure 10: Distillation flow: the teacher generates soft probability maps and the student learns from both the real mask and the teacher.

7.3 Why soft labels help

Hard labels say only 0 or 1. Teacher probabilities can express uncertainty. For example, a boundary pixel might be 0.58 rather than a hard 1. This helps the student learn smoother boundaries and less brittle decisions.

7.4 Fairness-aware loss

Fair KD A uses group weighting. A simple way to explain it:

$$L_{KD,A} = \frac{1}{B} \sum_i w_{g_i} \cdot BCE(z_{S,i}, P_{T,i}),$$

where g_i is the ITA group for image i , and w_{g_i} is larger for groups with lower validation Dice.

Fair KD B adds feature distribution alignment:

$$L_{KD,B} = \alpha L_{sup} + (1 - \alpha) L_{KD,A} + \lambda_{mmd} \frac{1}{G} \sum_g MMD(T_g, S_g).$$

In easy language: Approach A changes the importance of examples by group. Approach B also tries to make the student's internal representations look like the teacher's representations within each group.

8 Temperature scaling

8.1 What temperature scaling does

A model often outputs logits that are too confident. Temperature scaling modifies logits before sigmoid:

$$p = \sigma\left(\frac{z}{T}\right).$$

- $T = 1$: no scaling.
- $T > 1$: softer probabilities, less extreme confidence.
- $T < 1$: sharper probabilities, more extreme confidence.

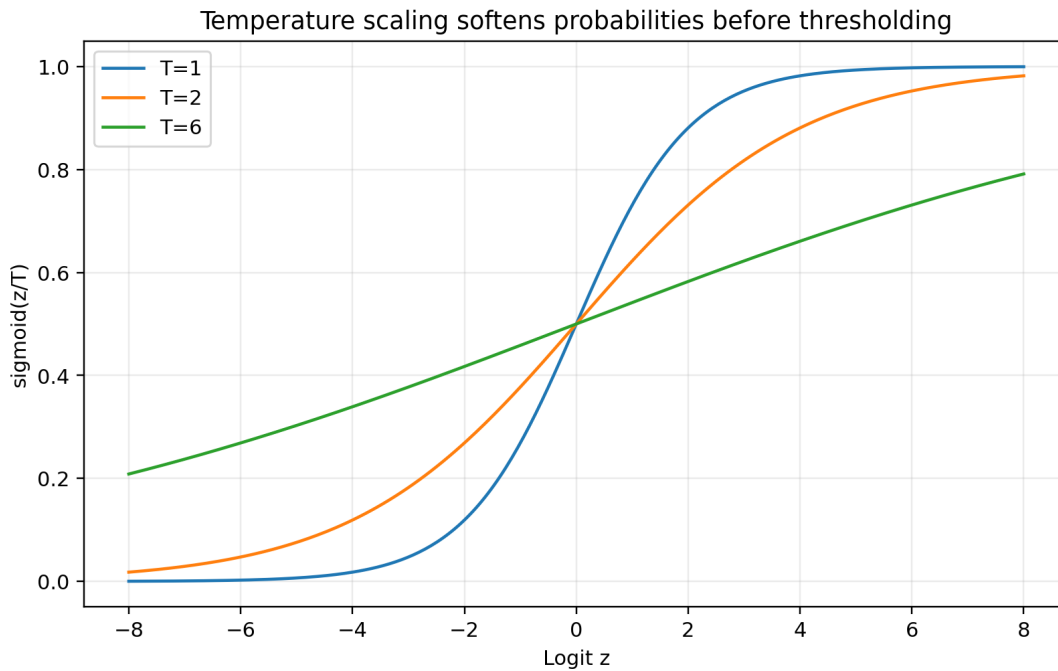


Figure 11: Temperature scaling softens probabilities. Higher temperature makes the sigmoid curve flatter, which gives the student more graded soft-label information.

8.2 Why teacher temperature is useful

If the teacher outputs probabilities very close to 0 or 1 everywhere, the soft label becomes almost the same as a hard label. Temperature scaling makes the teacher output more informative probabilities, especially near boundaries and uncertain areas.

8.3 Why YOLO temperature results are currently under audit

The YOLO audit shows:

- raw logits without temperature: low mean Dice;
- raw logits with temperature: improved mean Dice;
- native Ultralytics prediction: much higher mean Dice.

This means the raw path is probably not equivalent to the native path. The likely cause is preprocessing mismatch, so the YOLO threshold and temperature results must be rerun before final claims.

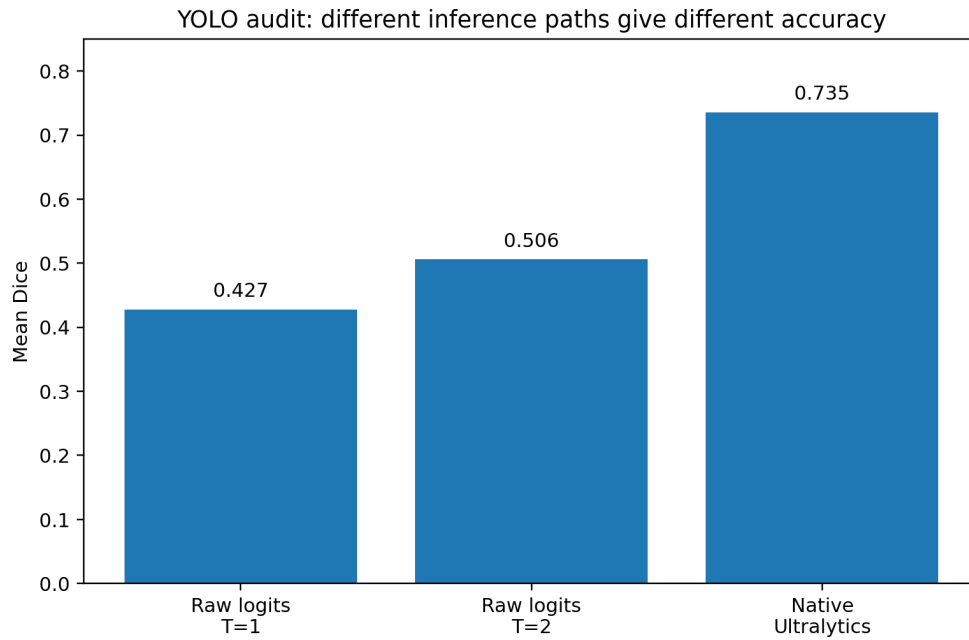


Figure 12: YOLO direct audit: native Ultralytics prediction performs much better than the custom raw-logit path. This is the main reason YOLO is marked unresolved.

9 Thresholding

9.1 Why thresholding is necessary

Segmentation models output probabilities. A final mask requires a binary decision per pixel:

$$\hat{Y}_{ij} = \mathbb{K}[p_{ij} \geq \tau],$$

where τ is the threshold.

9.2 How threshold affects precision and recall

- Lower threshold: predicts more bruise pixels. Recall usually increases, but false positives can increase.
- Higher threshold: predicts fewer bruise pixels. Precision usually increases, but recall can fall.

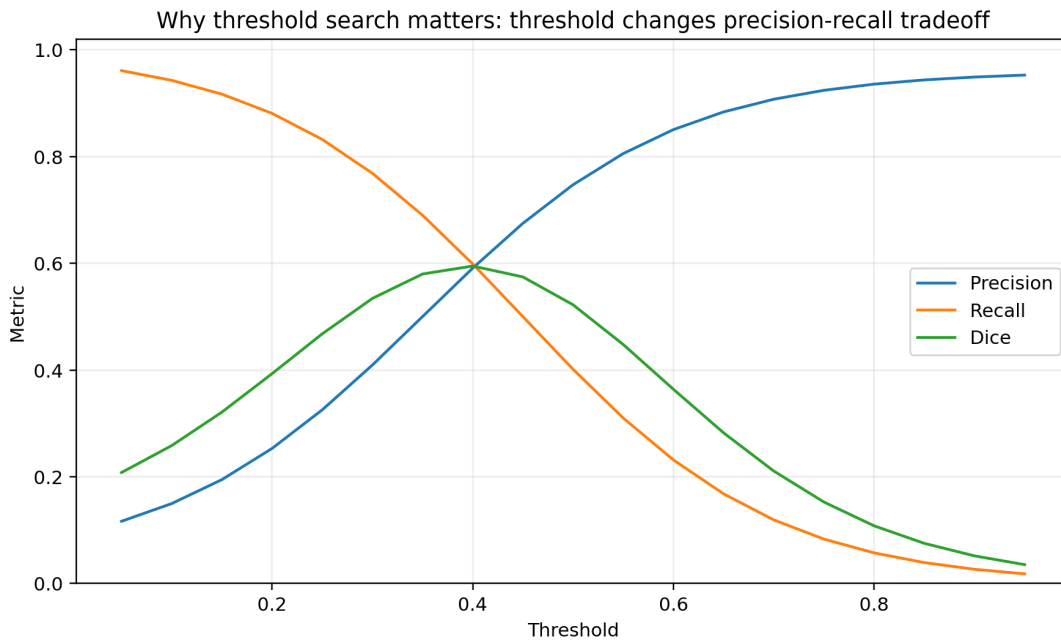


Figure 13: Conceptual threshold sweep. The best Dice is often not at 0.50 because threshold changes the precision-recall balance.

9.3 Validation threshold search protocol

The correct protocol is:

1. Train the model on the training split.
2. Evaluate candidate thresholds on validation data.
3. Pick the threshold with the best validation Dice.
4. Freeze that threshold.
5. Evaluate once on the fixed test set.

This protects the test set from leakage.

10 Evaluation metrics

10.1 Pixel counts

For each predicted mask and ground-truth mask:

- True positives (TP): predicted bruise and actually bruise.
- False positives (FP): predicted bruise but actually background.
- False negatives (FN): predicted background but actually bruise.

10.2 Dice

$$Dice = \frac{2TP}{2TP + FP + FN}.$$

Dice is the most important metric here because it directly measures overlap between predicted and true masks.

10.3 IoU

$$IoU = \frac{TP}{TP + FP + FN}.$$

IoU is stricter than Dice. For the same prediction, IoU is usually lower than Dice.

10.4 Precision and recall

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}.$$

Precision answers: of the pixels predicted as bruise, how many are actually bruise? Recall answers: of the true bruise pixels, how many did the model find?

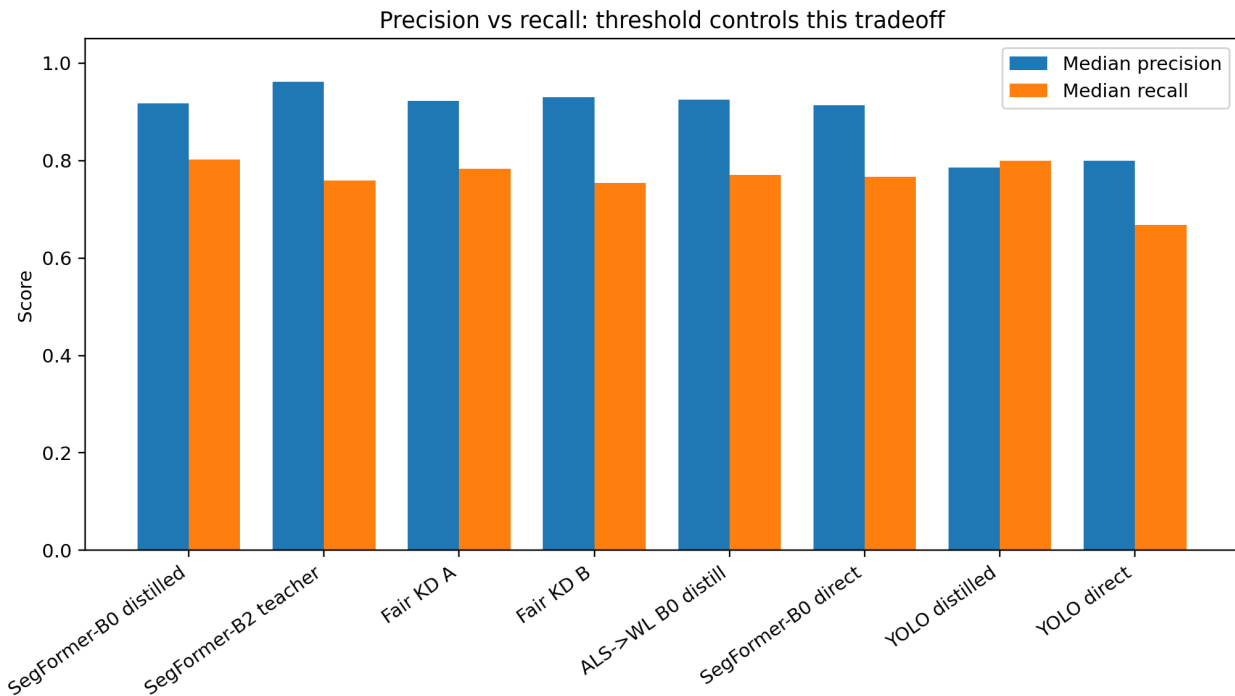


Figure 14: Median precision and recall by model. Threshold choices influence this tradeoff strongly.

10.5 Zero Dice and complete miss

Zero Dice means there is no overlap between prediction and ground truth. **Complete miss** means the ground truth has bruise pixels but the model predicts zero positive pixels. Complete miss is more clinically concerning because the model entirely missed the bruise.

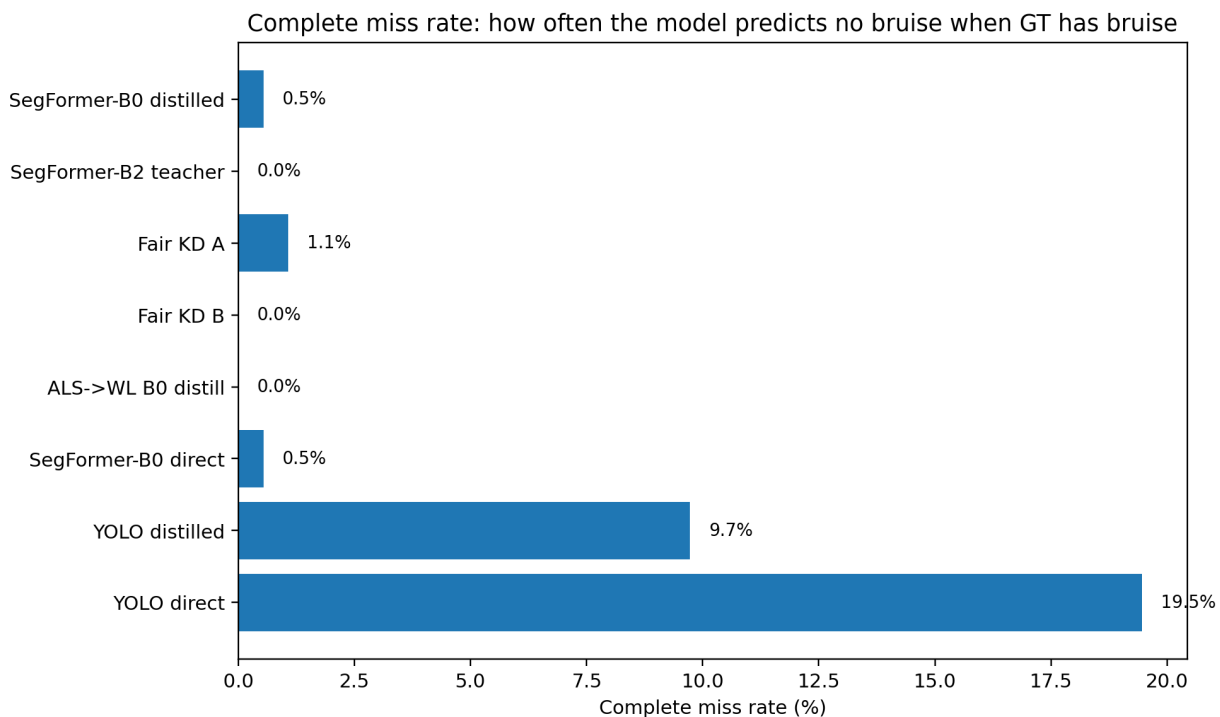


Figure 15: Complete miss rate by model. Lower is better. B2 and Fair KD B have 0 percent complete miss in the uploaded fixed-test summary, while B0 distilled has a very low miss rate.

10.6 Surface Dice, ASD, and HD95

Standard Dice measures region overlap. Boundary metrics measure whether the predicted edge is close to the true edge.

- **Surface Dice:** boundary agreement within a tolerance.
- **ASD:** average surface distance in pixels.
- **HD95:** 95th percentile Hausdorff distance, a robust boundary worst-case measure.

These metrics are useful when two masks overlap similarly but one has a worse boundary.

11 Evaluation resolution and resizing

11.1 Why images and masks are resized

Neural networks are trained and evaluated on fixed tensor sizes, here 640x640. The image is resized to 640x640 and the ground-truth mask is resized to the same grid. This lets the model prediction and ground truth be compared pixel-to-pixel.

11.2 Why nearest-neighbor is used for masks

Image resizing can use bilinear interpolation because RGB images are continuous. Binary masks should use nearest-neighbor resizing because a mask should remain 0 or 1. Bilinear interpolation would create

fractional labels unless binarized again.

11.3 Evaluation vs deployment resizing

There are two different resizing contexts:

- **Evaluation resizing:** compare prediction and GT at the same 640x640 grid.
- **Deployment resizing:** after thresholding, resize the binary mask back to the original camera image size for overlay.

If asked why resizing is not wrong: the model must operate on a fixed tensor size, and comparison is fair because both prediction and ground truth are on the same grid. For real output, the mask is returned to original image dimensions.

12 Benchmarking

12.1 Raw forward vs full pipeline

The benchmark reports two FPS values:

- **Raw forward FPS:** only the model's neural network forward pass. The tensor is already on GPU and preprocessed.
- **Full pipeline FPS:** deployment-style path: image in RAM, resize, normalize/scale, move to GPU, model forward, probability conversion, thresholding, and resize mask to original resolution.

Full-pipeline benchmark definition

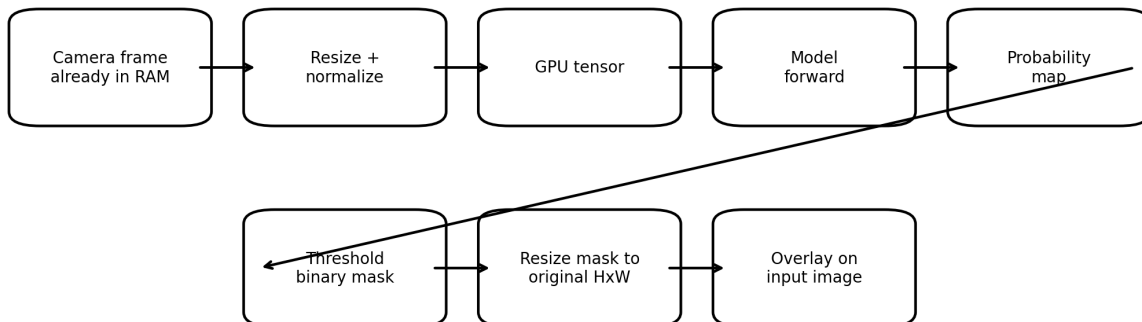


Figure 16: Full-pipeline benchmark definition. This matches the supervisor's camera-to-overlay-mask requirement.

12.2 Why disk reading is excluded

Disk image reading is not timed because a deployed camera system receives a frame already in memory. Disk read speed depends on filesystem caching and storage, not model deployment speed.

12.3 Why warmup and CUDA synchronization are needed

GPU operations are asynchronous. Without `torch.cuda.synchronize`, timing can measure only CPU launch time. Warmup prevents first-iteration overhead from distorting steady-state speed.

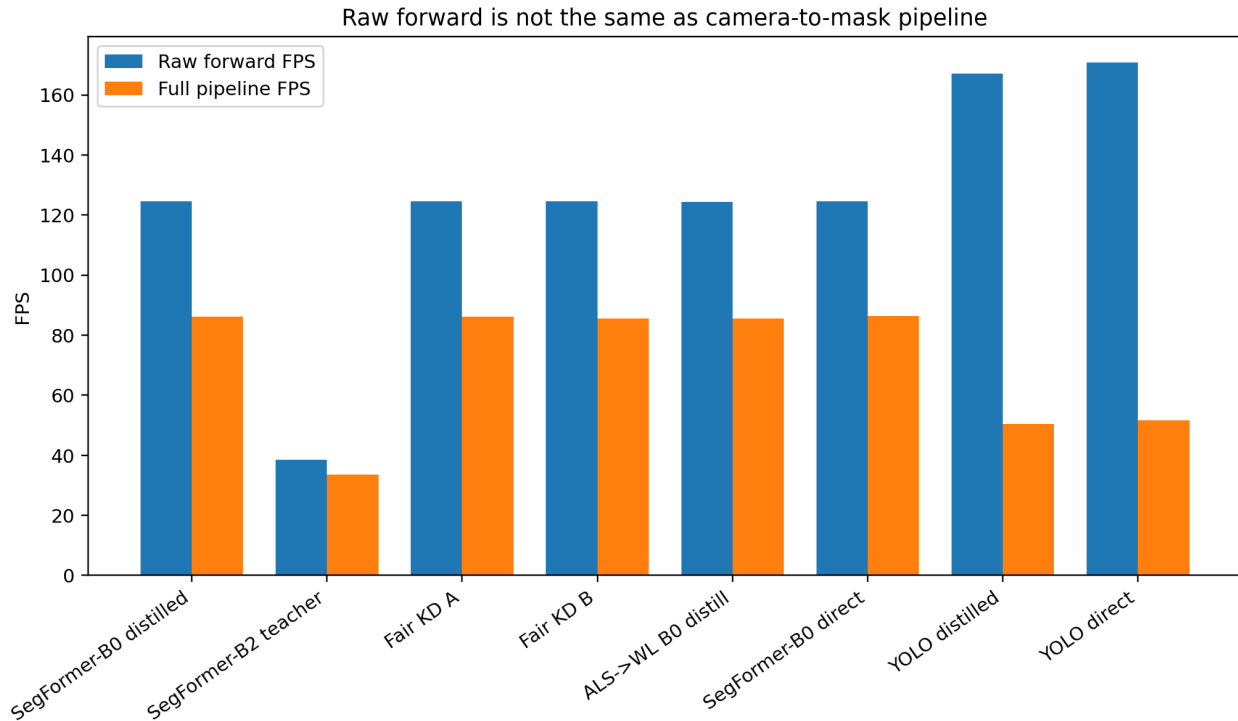


Figure 17: Raw forward FPS is higher than full-pipeline FPS because it excludes preprocessing and postprocessing.

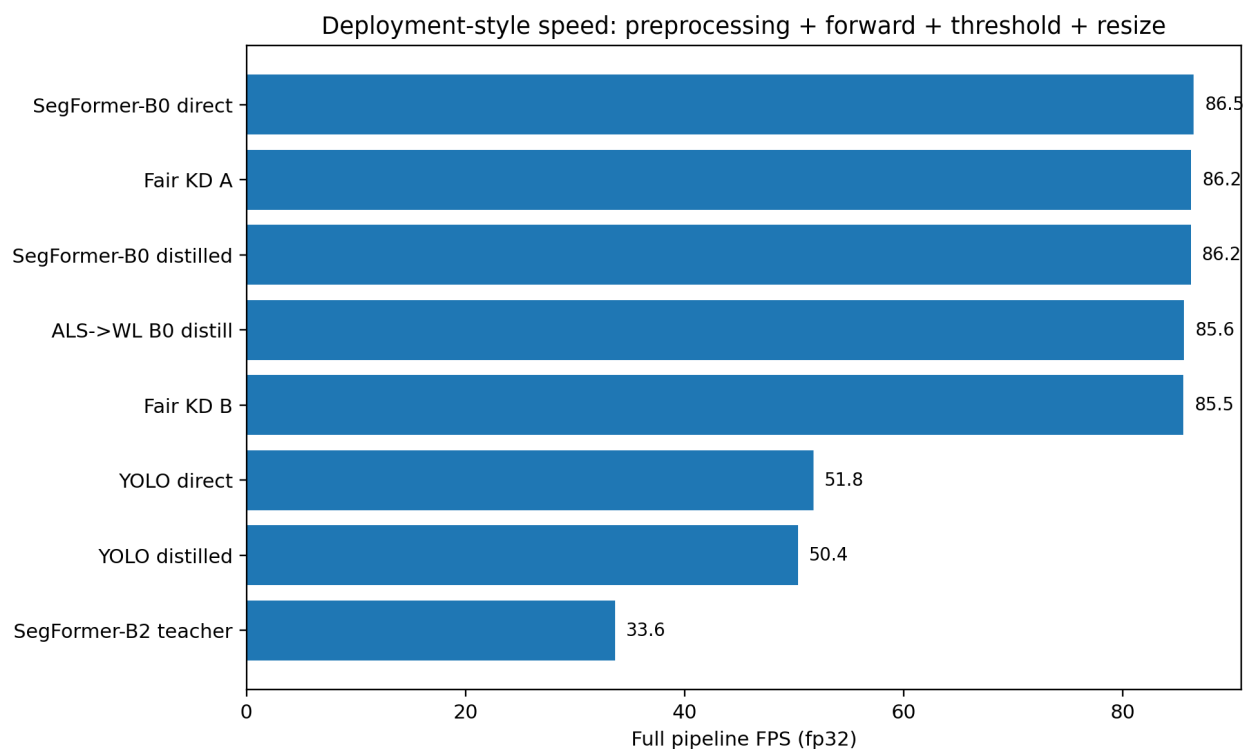


Figure 18: Full-pipeline fp32 FPS. B0 variants run around 85–86 FPS, while B2 is much slower.

12.4 fp32 vs fp16

fp32 uses 32-bit floating point. fp16 uses 16-bit floating point. fp16 can be faster on some GPUs because tensor cores accelerate half-precision operations, but speed is not guaranteed. In this benchmark, fp16 improved B2 full-pipeline FPS but was slower for B0 and YOLO full-pipeline FPS.

Table 6: Detailed benchmark table. Raw forward isolates model architecture speed; full pipeline is the deployment-style number.

Model	Dtype	Thr.	Temp.	Raw FPS	Full FPS	Raw ms	Full ms
ALS->WL B0 distill	fp16	0.35	–	105.9	76.4	9.44	13.10
ALS->WL B0 distill	fp32	0.35	–	124.4	85.6	8.04	11.69
Fair KD A	fp16	0.30	–	107.7	76.9	9.29	13.00
Fair KD A	fp32	0.30	–	124.5	86.2	8.03	11.60
Fair KD B	fp16	0.35	–	106.1	76.4	9.43	13.09
Fair KD B	fp32	0.35	–	124.5	85.5	8.03	11.69
SegFormer-B0 direct	fp16	0.60	–	108.0	77.8	9.26	12.85
SegFormer-B0 direct	fp32	0.60	–	124.6	86.5	8.03	11.56
SegFormer-B0 distilled	fp16	0.55	–	107.9	76.9	9.27	13.01
SegFormer-B0 distilled	fp32	0.55	–	124.6	86.2	8.03	11.60
SegFormer-B2 teacher	fp16	0.75	–	48.8	41.5	20.48	24.10
SegFormer-B2 teacher	fp32	0.75	–	38.5	33.6	26.00	29.72
YOLO direct	fp16	0.10	2.0	139.1	46.0	7.19	21.76
YOLO direct	fp32	0.10	2.0	170.9	51.8	5.85	19.32
YOLO distilled	fp16	0.40	6.0	139.2	46.5	7.18	21.51
YOLO distilled	fp32	0.40	6.0	167.2	50.4	5.98	19.86

13 Result interpretation

13.1 Main accuracy result

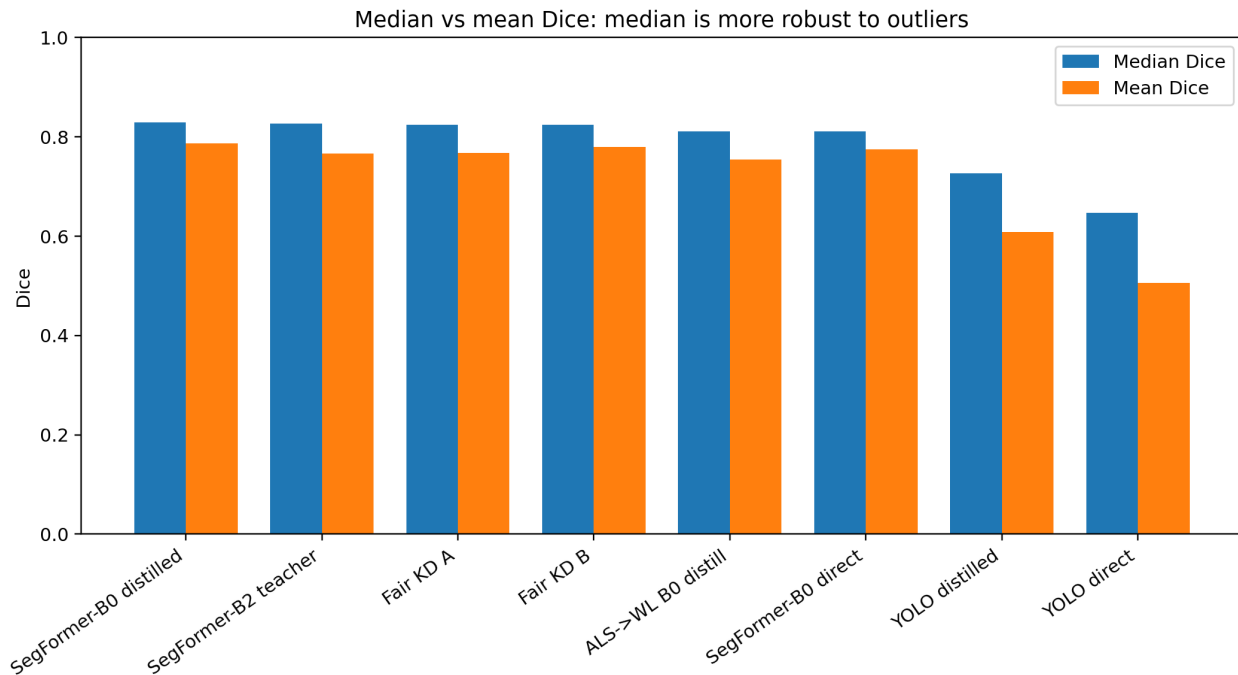


Figure 19: Median and mean Dice. When mean is lower than median, it usually means a subset of hard cases or misses pulls the average down.

The uploaded fixed-test summary shows that SegFormer-B0 distilled has the highest median Dice. B2 teacher is extremely close, but slower. Fairness variants are close in median Dice and may improve group robustness, but they do not clearly beat B0 distilled in overall accuracy.

13.2 Why B0 distilled is the best deployment answer

B0 distilled is the strongest current deployment model because:

- it has the best median Dice in the fixed-test results;
- it is much faster than B2 teacher in the full pipeline;
- it maintains very low complete-miss rate;
- it demonstrates the intended value of distillation: teacher-like accuracy with student-like speed.

13.3 Why B2 teacher is still important

B2 teacher is not useless just because it is slower. It provides the supervision signal for distillation. It is also the high-capacity reference model that shows what the larger architecture can learn.

13.4 Why YOLO should be discussed carefully

The current YOLO story is not simply “YOLO is bad.” The correct story is:

YOLO’s native Ultralytics path gives much stronger performance than the custom raw-logit temperature path. This suggests an evaluation-path mismatch, most likely preprocessing. Therefore YOLO must be rerun with corrected preprocessing before final comparison.

This is a strength in a defense: it shows you audited the system instead of blindly trusting every number.

14 Fairness and ITA evaluation

14.1 What fairness means in this project

Fairness here means performance consistency across ITA skin-tone groups. It does not prove global demographic fairness, but it checks whether segmentation quality changes substantially across measured skin-tone categories.

14.2 Fairness gap

A simple fairness gap is:

$$Gap = \max_g MedianDice_g - \min_g MedianDice_g.$$

Lower gap means more consistent median performance across groups.

Table 7: Derived median-Dice fairness gap across ITA groups from available per-image outputs. Lower is better. YOLO values are still affected by the YOLO audit caveat.

Model	Gap	Worst group median	Best group median
Fair KD B	0.026	0.809	0.836
Fair KD A	0.073	0.800	0.873
SegFormer-B0 distilled	0.113	0.756	0.869
ALS->WL B0 distill	0.138	0.721	0.859
SegFormer-B2 teacher	0.145	0.731	0.877
SegFormer-B0 direct	0.152	0.696	0.848
YOLO distilled	0.317	0.514	0.831
YOLO direct	0.753	0.016	0.768

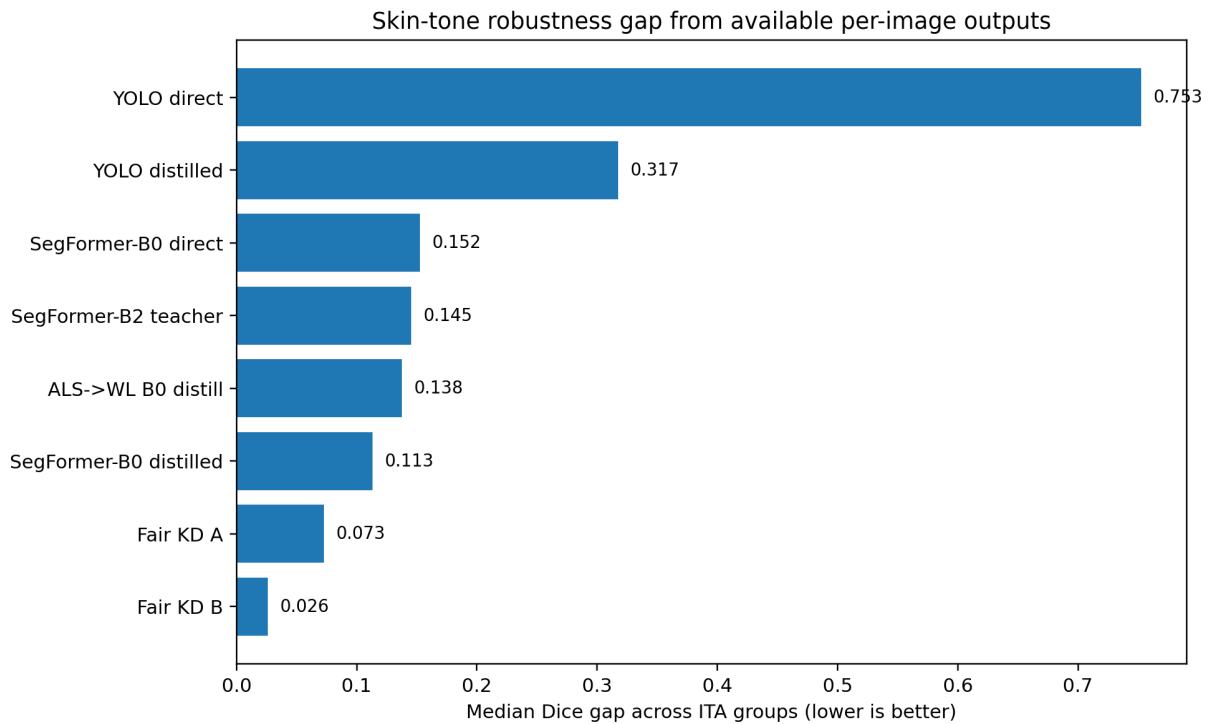


Figure 20: Fairness gap across ITA groups. Fair KD B has the smallest derived gap among the uploaded all-model per-image outputs.

14.3 How to answer fairness questions

A careful answer is:

We computed ITA-based skin-tone groups and checked performance across groups. The fixed test set had full ITA coverage. Fairness-aware distillation, especially Approach B, reduced the median-Dice gap across groups. However, fairness conclusions should be presented as evidence from this dataset, not as a universal guarantee.

15 Audit findings and what should be fixed before final publication

15.1 What looks solid

- SegFormer training/evaluation logic is coherent.
- Thresholds are selected on validation, not test.
- Fixed test evaluation is clearly separated.
- Full-pipeline benchmark matches the camera-to-mask deployment requirement.
- ITA coverage and subject leakage checks were run.
- B0 distilled gives a clean accuracy-speed story.

15.2 What needs caution

- YOLO raw-logit threshold/temperature results likely need a corrected YOLO preprocessing path.
- The all-model fairness script skipped some SegFormer checkpoints in the supplied log, so the official fairness folder does not include every model. Derived fairness can be computed from per-image test outputs, but a clean rerun is better.
- Some older consolidated CSV rows mix earlier native YOLO outputs with later 640 no-upscale SegFormer outputs. Do not treat that CSV as the single source of truth.

15.3 Exact YOLO fix

Create a YOLO-specific evaluation dataset or transform:

```
A.Compose([
    A.Resize(height=640, width=640),
    ToTensorV2(),
])
x = image_tensor.float() / 255.0
```

Do not apply ImageNet mean/std normalization to YOLO unless the YOLO training path used that exact same normalization. Then rerun:

1. YOLO direct threshold/temperature search on validation;
2. YOLO distilled threshold/temperature search on validation;
3. fixed-test evaluation at 640;
4. YOLO fairness evaluation;
5. benchmark if thresholds or postprocessing changed.

16 How to present the results

16.1 Best 60-second explanation

Our goal was to build a WL bruise segmentation model that outputs an overlayable mask. We trained a larger SegFormer-B2 teacher, then distilled it into smaller deployment models. We selected thresholds only on validation and evaluated on a fixed 185-image test set. SegFormer-B0 distilled achieved the best median Dice while running around 86 FPS in the full deployment-style benchmark, compared with about 34 FPS for B2. This means distillation preserved teacher-level accuracy while improving deployment speed. We also evaluated ITA skin-tone fairness and found that fairness distillation, especially Approach B, reduced group performance gaps. YOLO is still under audit because its native and raw-logit paths disagree, likely due to preprocessing mismatch.

16.2 Best 15-second conclusion

The best current deployment model is SegFormer-B0 distilled: it gives the best typical Dice, very low miss rate, and more than twice the full-pipeline speed of the B2 teacher. YOLO needs a corrected preprocessing rerun before final comparison.

17 Script map

Table 8: What each uploaded script does and how it fits into the pipeline.

Script	Role
00_build_split.py	Builds train/validation splits from the manifest. This is where subject-level separation must be enforced.
01_train_segformer_b2_teacher.py	Trains the high-capacity SegFormer-B2 teacher on binary bruise masks.
02_calibrate_teacher.py	Finds a temperature value for the B2 teacher logits so the teacher produces softer probability maps for distillation.
03_train_segformer_b0_direct.py	Trains the small SegFormer-B0 directly against ground-truth masks using supervised loss.
04_optuna_alpha_segformer_b0.py	Searches the distillation weight alpha for SegFormer-B0 student training.
05_train_segformer_b0_distilled.py	Trains B0 student using both ground truth and teacher soft labels.
06_train_yolo_sem_direct.py	Trains the YOLO semantic model directly on bruise masks through Ultralytics.
07_optuna_alpha_yolo_sem.py	Searches distillation alpha for the YOLO student.
07b_threshold_yolo_direct.py	Sweeps threshold/temperature for YOLO direct. This path needs the YOLO-specific preprocessing correction noted in the audit.
07c_threshold_yolo_distilled.py	Sweeps threshold/temperature for YOLO distilled. This path needs the same YOLO preprocessing correction.
08_train_yolo_sem_distilled.py	Trains YOLO student from teacher signal plus ground truth.
09_evaluate_test.py	Older test evaluation script for the initial model set.
09b_evaluate_yolo_temp_scaled.py	Evaluates YOLO raw logits with temperature scaling; flagged because it uses the generic BruiseDataset path.
10_consolidate_results.py	Combines validation/test results for the first model group.
10_track_a_evaluate.py	Applies extended test metrics to SegFormer models.
11_track_b_evaluate.py	Applies extended test metrics to SegFormer and YOLO models.
12_train_segformer_b0_fairness_distill.py	Trains fair distillation variants using ITA groups, inverse-Dice weighting, and optional MMD feature alignment.
13_train_segformer_b2_als_teacher.py	Trains an ALS-input SegFormer-B2 teacher.
14_calibrate_als_teacher.py	Calibrates temperature for the ALS teacher.
15_optuna_alpha_als_to_wl.py	Searches alpha for distilling ALS teacher knowledge into a WL student.
16_train_segformer_b0_als_to_wl_distill.py	Trains a WL B0 student using soft knowledge transferred from an ALS teacher.

Script	Role
<code>17_fairness_eval_all_models.py</code>	Runs ITA group fairness evaluation. Some SegFormer models were skipped due to checkpoint compatibility in the supplied log.
<code>18_compare_distillation_strategies.py</code>	Compares the different distillation strategies.
<code>19_compute_ita_train.py</code>	Computes ITA labels for WL training images.
<code>20_compute_ita_test.py</code>	Computes ITA labels for the WL fixed test images.
<code>21_assign_als_ita.py</code>	Maps ALS images to available WL ITA labels.
<code>22_add_ita_to_train_split.py</code>	Adds ITA group columns to the train/validation split.
<code>23_verify_ita_coverage.py</code>	Verifies ITA coverage and leakage checks.
<code>24_consolidate_all_results.py</code>	Consolidates later result groups.
<code>25_evaluate_als_teacher_test.py</code>	Evaluates the ALS teacher on test data.
<code>26_migrate_legacy_segformer_checkpoints.py</code>	Migrates older SegFormer checkpoint keys to the current wrapper format.
<code>27_evaluate_test_640_models_01_to_11.py</code>	Evaluates models 01-11 at 640x640 using no upscaling of GT; YOLO portion should be rerun with correct YOLO preprocessing.
<code>28_evaluate_test_640_models_12_plus.py</code>	Evaluates later SegFormer models such as fairness and ALS-to-WL variants.
<code>29_benchmark_inference_all_models.py</code>	Benchmarks raw forward and full camera-to-overlay pipeline speed for all WL-deployable models.

18 Q&A defense guide

The following answers are written in simple language so they can be used directly during a meeting, presentation, or thesis/paper discussion.

What exactly is the problem being solved?

Binary semantic segmentation of bruises in white-light skin images. For every input image, the model must output a same-aspect-ratio binary mask where bruise pixels are 1 and background pixels are 0.

Why is this semantic segmentation and not classification?

Classification would only say whether an image has a bruise. Semantic segmentation gives a pixel-level map, which is needed to measure bruise area, shape, boundary, and overlay the result on the original image.

Why did we use SegFormer?

SegFormer is strong for dense prediction because its transformer encoder captures context at multiple scales and its lightweight MLP decoder fuses features without a heavy convolutional decoder. It is accurate and relatively easy to adapt to one-class binary masks.

What is the difference between B2 and B0?

Both are SegFormer variants, but B2 is larger and more accurate/expensive, while B0 is smaller and faster. In this run, B2 has about 27.35M parameters and B0 has about 3.71M parameters.

Why train a B2 teacher first?

The larger teacher can learn richer segmentation behavior. It can then produce soft probability maps that help the smaller student learn not only the hard labels but also the teacher confidence structure.

What is knowledge distillation in this project?

A smaller student model is trained with a combined loss: one part uses the real ground-truth mask, and the other part asks the student to match the teacher probability map.

What does alpha mean?

Alpha controls the balance between ground-truth supervision and teacher imitation. If alpha is high, the student trusts the ground truth more. If alpha is low, it trusts the teacher more.

Why not use only the teacher labels?

The teacher is not perfect. Ground truth anchors the student to the true labels, while teacher probabilities provide extra signal about uncertain pixels and boundary regions.

What is thresholding?

The model outputs probabilities, not binary masks. Thresholding converts probability maps to binary masks: pixels above the threshold become bruise, pixels below become background.

Why not always use threshold 0.50?

A probability of 0.50 is a generic default, not necessarily the best operating point for this dataset. Validation threshold search chooses the threshold that maximizes validation Dice.

Why is threshold selected on validation and not test?

Choosing threshold on the test set would leak test information and inflate reported performance. The test set must be used only after model and threshold are fixed.

What is temperature scaling?

Temperature scaling divides logits by T before sigmoid. Higher T softens extreme logits, making probabilities less overconfident. It is useful for distillation and sometimes for threshold calibration.

Is temperature scaling training?

No. Temperature scaling is a calibration/post-processing step unless explicitly integrated into the training loss. In this pipeline, teacher temperature helps produce soft labels; YOLO temperature was used in threshold/evaluation calibration.

Why did temperature scaling not clearly beat argmax/native YOLO?

For YOLO, the audit found that the raw-logit evaluation path likely used SegFormer-style ImageNet normalization instead of YOLO 0-1 scaling. That makes the raw-logit temperature results unreliable until rerun.

What is Dice?

Dice measures overlap between predicted bruise mask and ground truth: two times intersection divided by predicted area plus ground-truth area. It is the primary segmentation metric here.

What is IoU?

IoU is intersection divided by union. It is stricter than Dice for the same prediction error.

Why use median Dice?

Per-image Dice is skewed by difficult examples and complete misses. Median better describes the typical image, while mean is still useful to expose outliers.

What is complete miss rate?

The percentage of images where the ground truth has bruise pixels but the model predicts no positive bruise pixels. It is clinically important because a total miss is worse than a boundary error.

What is zero-Dice rate?

The percentage of images with Dice equal to zero. It includes complete misses and cases with no overlap.

What is pred/GT ratio?

Predicted positive pixel count divided by ground-truth positive pixel count. Values below 1 mean under-segmentation; values above 1 mean over-segmentation.

Why resize ground truth to 640?

The model operates at a fixed tensor size. During fair apple-to-apple evaluation, both prediction and ground truth are compared at the same 640x640 grid. For deployment overlay, the final mask is resized back to the original camera image size.

Is resizing the mask wrong?

No, as long as prediction and ground truth are resized consistently for evaluation, and nearest-neighbor interpolation is used for binary masks. The final deployment mask must be resized back to the original image shape for overlay.

What is raw forward FPS?

Only the neural network forward pass is timed. The tensor is already preprocessed and already on GPU. This isolates architecture speed.

What is full pipeline FPS?

The deployment-style timing: image in RAM to resized normalized tensor, GPU transfer, model forward, sigmoid or temperature, threshold, and resize mask back to original size.

Which FPS should I present?

Present full pipeline FPS as the main deployment number. Raw forward FPS is useful for architecture comparison but does not represent end-to-end camera-to-mask speed.

Why is full pipeline FPS lower than raw forward FPS?

Because it includes preprocessing, transfer, thresholding, CPU mask extraction, and resizing, not just the model.

Why is fp16 not always faster?

fp16 speed depends on GPU kernels, overhead, model size, and preprocessing bottlenecks. B2 improved with fp16, but B0 and YOLO were faster in fp32 in this benchmark.

Which model is best overall right now?

SegFormer-B0 distilled. It has the best median Dice in the fixed test evaluation and a much higher full-pipeline FPS than B2.

Why not choose B2 teacher if it has zero miss rate?

B2 is accurate but slower. B0 distilled nearly matches or slightly exceeds B2 median Dice and runs more than twice as fast in the full pipeline.

What does Fair KD A do?

It reweights distillation loss by ITA skin-tone group so underperforming groups receive more training emphasis.

What does Fair KD B add?

It adds per-group feature distribution alignment using MMD, encouraging the student to match the teacher internally as well as at the output map.

What is ITA?

Individual Typology Angle, a color-based skin-tone measurement computed from image regions. It is used to group images for fairness analysis.

Does the fairness experiment prove no bias?

No. It measures performance across available ITA groups. It is useful evidence, but fairness conclusions depend on sample size, data quality, and whether ITA groups represent the target population.

Why is YOLO flagged?

The native Ultralytics path and raw-logit path give very different numbers. The likely reason is preprocessing mismatch: raw-logit scripts use the generic ImageNet-normalized dataset path, while YOLO expects 0-1 scaled input.

Does this mean YOLO is unusable?

No. It means YOLO must be rerun with corrected preprocessing before making final claims. Native YOLO results are much better than the raw-logit results.

What is the safest final claim?

The most defensible deployment model from the uploaded run is SegFormer-B0 distilled. YOLO is still under audit until the raw-logit evaluation path is corrected.

What would you fix next?

Create a YOLO-specific dataset/evaluation loader with resize plus ToTensor plus divide by 255, rerun threshold search, rerun test evaluation, and rerun YOLO fairness.

How do I explain the benchmark to my supervisor?

Say that the full-pipeline section includes exactly the steps needed for camera input to overlayable mask output, and excludes unrelated setup such as disk reading and model loading.

Why not time disk image reading?

A real deployment receives a camera frame in memory. Disk read measures storage I/O and filesystem caching, not model deployment speed.

Why use `torch.cuda.synchronize`?

GPU work is asynchronous. Synchronization makes sure timing measures completed GPU work, not just CPU launch time.

Why warmup iterations?

The first iterations include one-time overheads such as kernel selection and memory caching. Warmup prevents those from distorting steady-state FPS.

18.1 Architecture and model-internal Q&A**What happens inside SegFormer in one sentence?**

SegFormer converts the image into multi-scale transformer features, fuses those features with a lightweight MLP decoder, and outputs one bruise logit per pixel.

What is the encoder doing?

The encoder extracts feature maps. Early features preserve edges and texture; deeper features capture larger context and semantic meaning.

What is the decoder doing?

The decoder converts encoder features back into a dense pixel-level prediction. It combines low-resolution semantic information with higher-resolution spatial information.

Why does SegFormer use multi-scale features?

Bruises can be small, large, faint, or irregular. Multi-scale features let the model use fine edge information and broad context at the same time.

Why does the final SegFormer head output one channel?

This is binary segmentation. One channel is enough: after sigmoid, values near 1 mean bruise and values near 0 mean background.

What is a logit?

A logit is the raw model output before sigmoid. It is not limited to 0–1. Sigmoid converts it into a probability-like value.

Why not train directly on probabilities?

Losses such as BCE-with-logits are numerically more stable when they operate directly on logits. Sigmoid is still used for Dice computation, thresholding, and interpretation.

What exactly does the B2 teacher teach the B0 student?

It teaches a probability map. Instead of saying only “this pixel is bruise” or “not bruise,” the teacher gives graded confidence, which is useful around uncertain boundaries.

Why does distillation help if the student architecture is unchanged?

The architecture is unchanged, but the training signal is richer. The student learns from both human/consensus masks and the teacher’s softer boundary/confidence behavior.

What is the internal difference between B0 direct and B0 distilled at inference time?

There is almost no inference-time architectural difference. Both are B0 SegFormer models. The difference is how their weights were learned during training.

What is the internal difference between Fair KD A and normal KD?

Fair KD A changes the loss weighting per ITA group. Underperforming groups receive a stronger teacher-matching penalty.

What is the internal difference between Fair KD B and Fair KD A?

Fair KD B also aligns hidden decoder feature distributions using MMD. So it tries to match internal representation, not just output probability.

Why use MMD instead of just comparing feature vectors with MSE?

MSE compares corresponding vectors directly. MMD compares distributions of features. For fairness groups, distribution matching is more appropriate because we care whether the student’s group-level representation resembles the teacher’s group-level representation.

What does YOLO output internally?

In the raw semantic path, YOLO outputs class logits over spatial positions. For two classes, channel 0 is background and channel 1 is bruise.

Why does the YOLO raw path use bruise logit minus background logit?

For two-class softmax, the probability of class 1 can be written as sigmoid of the logit difference: $p = \sigma(z_1 - z_0)$. This converts two class logits into one bruise probability map.

Why is YOLO preprocessing so important?

Because model weights learn a specific input distribution. YOLO was trained through Ultralytics-style image scaling, so feeding ImageNet-normalized tensors can shift the input distribution and damage raw-logit outputs.

What is the simple difference between SegFormer and YOLO?

SegFormer is transformer-based and uses a custom dense segmentation training/evaluation loop. YOLO is a fast Ultralytics architecture trained through its own semantic segmentation pipeline. SegFormer is currently the cleaner final result; YOLO needs a corrected raw-logit evaluation rerun.

What is the simple difference between SegFormer and DeepLabV3+?

SegFormer uses transformer attention and an MLP decoder. DeepLabV3+ uses a convolutional backbone, ASPP dilated convolutions, and a decoder with low-level skip features.

Why is nearest-neighbor interpolation used for final binary mask resizing?

A binary mask contains class labels, not continuous colors. Nearest-neighbor preserves 0/1 labels. Bilinear interpolation could create fractional mask values and blur boundaries.

19 Final recommendations

19.1 Use this as the final model conclusion for now

Recommended conclusion: SegFormer-B0 distilled is the best current deployment model because it achieves the strongest median Dice on the fixed test set while maintaining high full-pipeline FPS.

19.2 Do this before calling the whole project final

1. Rerun YOLO threshold search and test evaluation with YOLO-specific preprocessing.
2. Rerun YOLO fairness after the preprocessing fix.
3. Rerun the all-model fairness script after resolving skipped SegFormer checkpoint compatibility.
4. Keep one canonical results folder/table so older native YOLO results and newer raw-logit results are not mixed accidentally.
5. For paper writing, report both median and mean, but make median the main ranking metric because per-image segmentation has skew and outliers.

19.3 Canonical final table to show

For a presentation or paper draft, the most defensible table should contain:

- model name;
- training type: direct, distilled, fairness, ALS-to-WL;
- validation-selected threshold;
- median Dice and mean Dice on fixed test;
- median IoU;

- complete miss rate;
- full-pipeline FPS;
- fairness gap if the fairness run is complete.

20 Appendix: formulas in one place

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Temperature scaling

$$p = \sigma\left(\frac{z}{T}\right)$$

Thresholding

$$\hat{Y}_{ij} = \mathbb{I}[p_{ij} \geq \tau]$$

Dice

$$Dice = \frac{2TP}{2TP + FP + FN}$$

IoU

$$IoU = \frac{TP}{TP + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

Supervised Dice-BCE loss

$$L_{sup} = BCE(z, Y) + (1 - Dice(\sigma(z), Y))$$

Distillation loss

$$L = \alpha L_{sup} + (1 - \alpha) BCE(z_S, P_T)$$

Fairness gap

$$Gap = \max_g MedianDice_g - \min_g MedianDice_g$$